

MIAE 215 · Embedded C++ & Arduino Mechatronics

Master Laboratory Manual & Term Project Guide

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)
Instructor: Prof. Brandon W. Gordon · Platform: Atmel ATmega328P / Arduino Uno · Language: Embedded C++

Table of Contents

- 1. [Academic Regulations & Laboratory Policies](#)
- 2. [Microcontroller Architecture & Hardware Specifications](#)
- 3. [Lab 1: Software Environment, USB Drivers & Execution Lifecycle](#)
- 4. [Lab 2: Digital Input/Output \(DIO\) & Execution Speed Profiling](#)
- 5. [Lab 3: Analog-to-Digital Conversion \(ADC\) & Noise Characterization](#)
- 6. [Lab 3 \(Advanced\): Pulse Width Modulation \(PWM\) & Servo Kinematics](#)
- 7. [Closed-Loop Control: The Autonomous Heliotropic Solar Tracker](#)
- 8. [Arduino Term Project Playbook & Sensor Integration](#)
- 9. [Final Exam Competency Blueprint & 10% High-Yield Questions](#)

1. Academic Regulations & Laboratory Policies

The laboratory component of **MIAE 215** provides hands-on practical experience with physical computing, digital sensor interfaces, real-time actuators, and embedded software architectures.

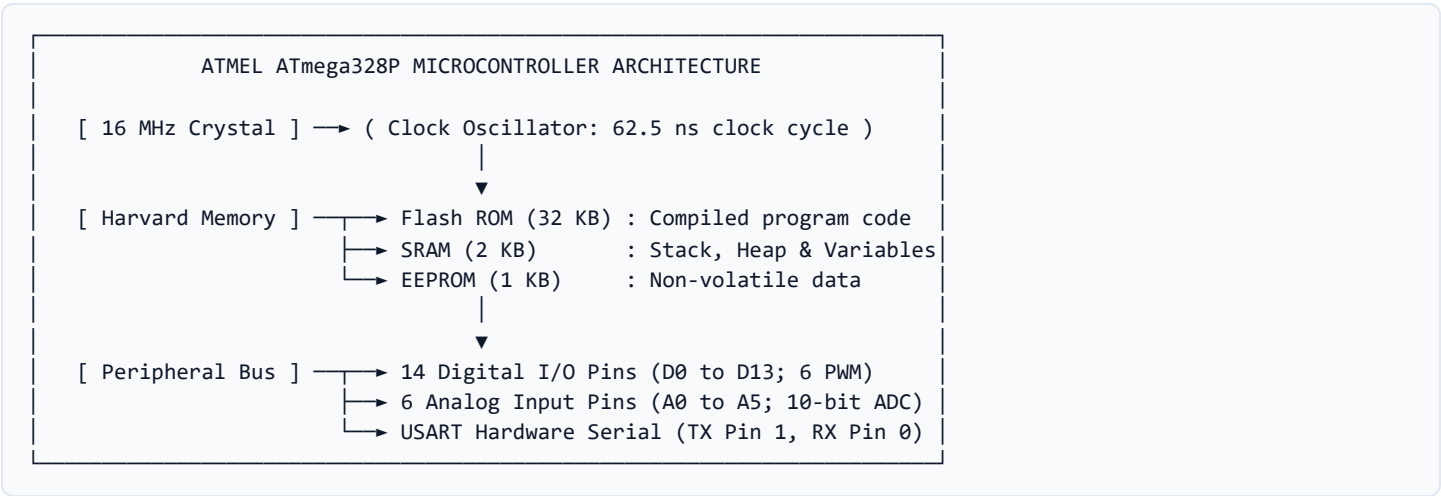
MIAE 215 LABORATORY TIMELINE & MILESTONES		
Period	Laboratory Module & Technical Scope	Submission Type
Week 4	Lab 1: IDE Setup, Drivers & Blink Test	Optional / Practice
Week 6	Lab 2: Digital I/O & micros() Timing	Optional / Practice
Week 8	Lab 3: Analog Sensors & Servo Actuators	Optional / Practice
Week 9+	Arduino Term Project Implementation	Graded (Moodle)
Final	Final Exam Questions (10% Course Total)	In-Person Exam

Core Course Rules

- **Independent Work:** Every student must work independently on the labs and the term project. Group work and code sharing are strictly prohibited.
- **Attendance Flexibility:** In-person attendance during scheduled lab slots is **completely optional**. All manuals, video demonstrations, and official solution programs are published online. In-person lab sessions serve as open support hours with Teaching Assistants (TAs).
- **Section Swapping:** Students may drop into other lab sections if space and physical workstations permit.
- **Grading Focus:** Only the **Arduino Term Project** is handed in for a coursework grade. However, **10% of the MIAE 215 Final Examination** consists of embedded programming and circuit questions drawn directly from Labs 1, 2, and 3.

2. Microcontroller Architecture & Hardware Specifications

In industrial, aerospace, and robotic engineering, a **mechatronic system** integrates mechanical structures, electronic sensors, microprocessors, and software. The **microcontroller** serves as the real-time embedded brain.



Hardware Specifications Table

Metric / Parameter	Value / Range	Engineering Implication
Processor Core	8-bit AVR RISC	Primitive <code>int</code> is 16-bit (<code>-32,768</code> to <code>+32,767</code>). No 64-bit hardware ALU.
Clock Frequency	16.0 MHz	Base instruction cycle duration is $T_{clk} = 62.5\text{ ns}$.
Operating Voltage	5.0 V	Logic HIGH $\approx 5.0\text{ V}$, Logic LOW $\approx 0.0\text{ V}$.
SRAM Memory	2,048 Bytes (2 KB)	Extreme memory constraints: large arrays cause stack collisions!
Flash Program ROM	32 KB (0.5 KB bootloader)	Stores compiled binary opcodes.
ADC Resolution	10-bit Successive Approx.	Maps $0\text{ V} - 5\text{ V}$ into integers $0 - 1023$ ($\approx 4.887\text{ mV/step}$).
Current per I/O Pin	20 mA (40 mA absolute max)	Never drive high-current motors directly from an I/O pin!

3. Lab 1: Software Environment, USB Drivers & Execution Lifecycle

Desktop C++ programs begin execution in a user-defined `main()` function and exit when finished. An embedded microcontroller never terminates; it powers on, runs initial hardware configuration once, and loops endlessly until power is disconnected.

```
#include <Arduino.h>

// setup() runs exactly ONCE upon power-up or hardware reset
void setup() {
  pinMode(13, OUTPUT); // Configure digital pin 13 as a push-pull output
}

// loop() executes repeatedly at the clock frequency of the processor
void loop() {
  digitalWrite(13, HIGH); // Drive pin 13 to +5V (LED ON)
  delay(1000);           // Busy-wait delay for 1000 milliseconds
  digitalWrite(13, LOW);  // Drive pin 13 to 0V (LED OFF)
  delay(1000);           // Busy-wait delay for 1000 milliseconds
}
```

Configuration & Upload Steps

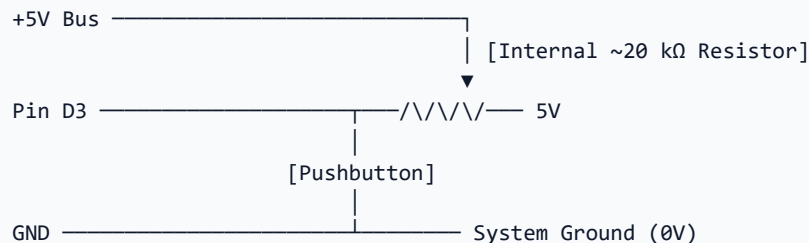
1. In the Arduino IDE, set **Tools** → **Board** → **Arduino AVR Boards** → **Arduino Uno**.
2. Select the designated virtual serial port under **Tools** → **Port** (**COMx** on Windows, **/dev/cu.usb...** on macOS).
3. Click the **Upload** arrow (o). The IDE invokes **avr-g++**, compiles the source into machine hex code, and transfers the binary through the onboard USB-to-serial bridge into Flash memory.

4. Lab 2: Digital Input/Output (DIO) & Execution Speed Profiling

Lab 2 teaches engineers how to handle discrete digital logic, configure input pull-up circuitry, stream timestamped telemetry, and measure execution speed.

Internal Pull-Up Resistor Architecture

Connecting an external mechanical switch directly between an input pin and +5V leaves the pin in an undefined, high-impedance state (**floating**) when unpressed. Electromagnetic noise will cause spurious readings. The ATmega328P incorporates internal $\sim 20\text{ k}\Omega$ pull-up resistors:



```
pinMode(3, INPUT_PULLUP); // Enables internal pull-up to +5V
```

- **Button OPEN (Released):** Pin D3 is pulled to +5.0V. `digitalRead(3)` returns **HIGH (1)**.
- **Button CLOSED (Pressed):** Current shunts to Ground. Pin D3 drops to 0.0V. `digitalRead(3)` returns **LOW (0)**.

Benchmarking Execution Speed via `micros()`

Because the ATmega328P lacks a hardware floating-point unit (FPU), calculating complex mathematical functions in software consumes substantial CPU time:

```

unsigned long t1 = micros(); // Capture microsecond timestamp before call
float val = sin(x);          // Perform floating-point calculation
unsigned long t2 = micros(); // Capture timestamp immediately after
unsigned long dt = t2 - t1;   // Compute delta time in microseconds

```

- On an 8-bit ATmega328P, a single `sin()` evaluation consumes ≈ 120 to $140, \mu\text{s}$!

Real-Time CSV Telemetry Streaming

To log experimental engineering data for plotting in MATLAB, Python, or Excel:

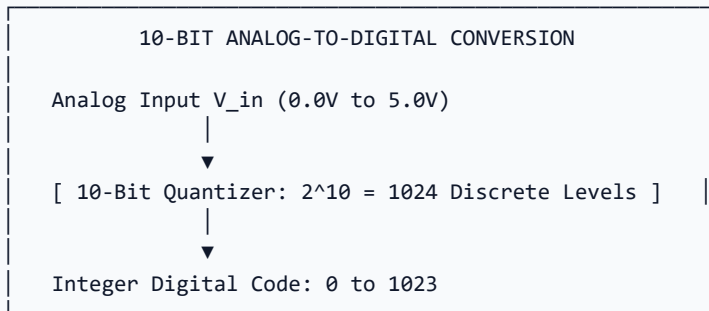
```

Serial.begin(115200); // Set high-speed communication
// Output comma-separated values: timestamp, voltage
Serial.print(micros() * 1.0e-6, 6);
Serial.print(",");
Serial.println(voltage);

```

5. Lab 3: Analog-to-Digital Conversion (ADC) & Noise Characterization

Microcontrollers cannot read continuously variable analog voltages directly. The Arduino Uno employs a 10-bit **Successive Approximation ADC** connected to pins A0 through A5.



Quantization Formula

$$\text{ADC Integer} = \text{round} \left(\frac{V_{in}}{V_{ref}} \times 1023 \right)$$

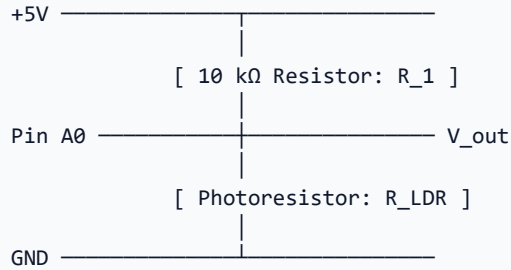
$$V_{in} = \frac{\text{analogRead}(A_i)}{1023.0} \times 5.0 \text{ V}$$

$$\text{Voltage Resolution} = \frac{5.0 \text{ V}}{1023} \approx 4.887 \text{ mV per LSB}$$

Voltage Divider Circuit for Light Sensing

A **Photoresistor (Light Dependent Resistor, LDR)** changes resistance with illumination:

- Dark resistance: $R_{LDR} \approx 100 \text{ k}\Omega - 1 \text{ M}\Omega$.
- Bright illumination: $R_{LDR} \approx 1 \text{ k}\Omega - 5 \text{ k}\Omega$.



$$V_{\text{out}} = 5.0 \text{ V} \times \frac{R_{\text{LDR}}}{R_1 + R_{\text{LDR}}}$$

Statistical Noise Profiling Algorithm (Lab 3 Q1)

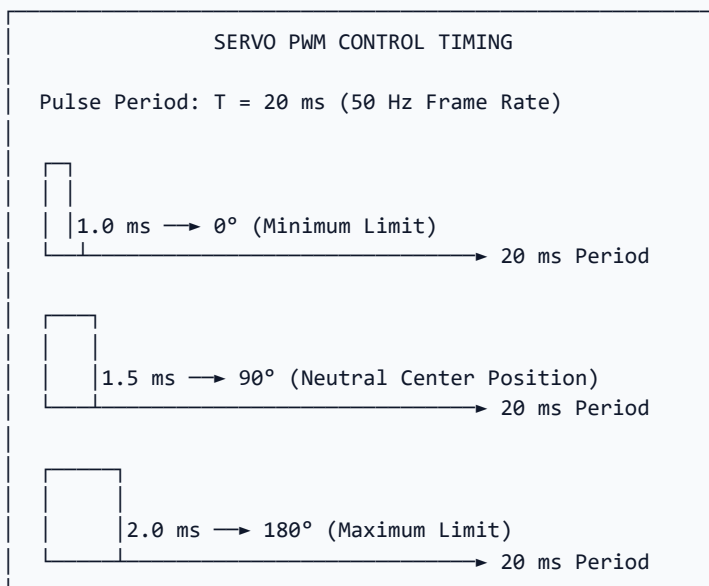
In physical environments, electronic signals exhibit electrical and optical noise:

```
float min_v = 1.0e6, max_v = -1.0e6, sum_v = 0.0;
const int N = 1000;

for (int i = 0; i < N; i++) {
    float v = analogRead(A0) * (5.0 / 1023.0);
    if (v < min_v) min_v = v;
    if (v > max_v) max_v = v;
    sum_v += v;
}
float ave_v = sum_v / N;
float delta_v = max_v - min_v; // Peak-to-peak noise voltage
```

6. Lab 3 (Advanced): Pulse Width Modulation (PWM) & Servo Kinematics

A standard hobby **micro-servo motor (SG90)** provides precision angular position control (0° to 180°) using an internal DC motor, gearbox, feedback potentiometer, and proportional error amplifier.



Driving Servos via `<Servo.h>`

```
#include <Servo.h>

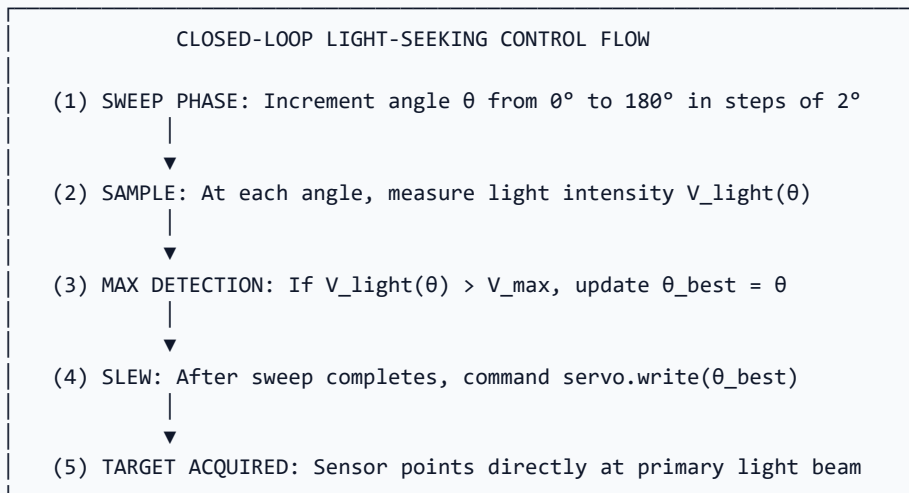
Servo my_servo; // Create servo instance

void setup() {
    my_servo.attach(7); // Binds pin D7 to servo control timer
}

void loop() {
    my_servo.write(90); // Commands output shaft to exactly 90 degrees
    delay(500);
    my_servo.write(180); // Commands output shaft to 180 degrees
    delay(500);
}
```

7. Closed-Loop Control: The Autonomous Heliotropic Solar Tracker

In **Lab 3 Question 3**, students combine sensory input with physical actuation to form an autonomous **closed-loop feedback system**. A photoresistor is physically mounted onto the rotating servo arm.



```
#include <Servo.h>

Servo tracker_servo;
const int SENSOR_PIN = A0;
const int SERVO_PIN = 7;

void setup() {
    tracker_servo.attach(SERVO_PIN);
    Serial.begin(115200);
}

void loop() {
    int best_angle = 0;
    float max_light_voltage = -1.0;

    // Phase 1: Spatial Sweep across hemisphere
    for (int angle = 0; angle <= 180; angle += 3) {
        tracker_servo.write(angle);
        delay(25); // Allow mechanical settling

        float light_val = analogRead(SENSOR_PIN) * (5.0 / 1023.0);
        if (light_val > max_light_voltage) {
            max_light_voltage = light_val;
            best_angle = angle;
        }
    }

    // Phase 2: Actuate to target optimum
    tracker_servo.write(best_angle);
    Serial.print("Target Locked at Angle: ");
    Serial.println(best_angle);

    delay(3000); // Dwell on target before scanning again
}
```

8. Arduino Term Project Playbook & Sensor Integration

The **Arduino Term Project** requires students to expand beyond Lab 3 by integrating a **new sensor** with the servo actuator to accomplish an automated, useful task.

APPROVED SENSOR MODULE BLUEPRINTS	
Sensor Module	Operational Physics & Project Idea
Ultrasonic (HC-SR04)	Distance ranging via speed of sound echo. Idea: Automated radar turret / parking sensor.
Dual-Axis Joystick	Analog 2-axis potentiometers (X, Y) + button. Idea: Manual robotic steering / pan-tilt aiming.
Temperature (TMP36)	Linear voltage output (10 mV/°C, 500 mV offset). Idea: Analog mechanical dial thermometer.

Moodle Submission Checklist

1. **Source Code** (`program.ino`): Clean, modular C++ implementation. Code must include comments explaining the algorithm (roughly one comment every 3–5 lines).
2. **Photos (1–2 images)**: Sharp photographs displaying the breadboard wiring, Arduino connections, and mechanical mounting.
3. **Demonstration Video (*.mp4)**: 1–2 minute video demonstrating the physical system operating autonomously.
4. **No Written Report Needed**: Your grade is assessed directly from code correctness, functionality, and originality.

9. Final Exam Competency Blueprint & 10% High-Yield Questions

The following topics account for **approximately 10% of the final exam**:

1. ADC Quantization & Voltage Mapping

- **Exam Question**: An Arduino analog pin reads `614`. What is the measured physical voltage?
- **Solution**:

$$V = \frac{614}{1023.0} \times 5.0 \text{ V} = \frac{3070}{1023} = 3.001 \text{ V}$$

2. Pull-Up Resistor Logic Inversion

- **Exam Question**: A switch is wired with `pinMode(2, INPUT_PULLUP)`. What does `digitalRead(2)` return when the user pushes the button down?
- **Solution**: When pressed, the circuit shunts to Ground (0V). Therefore, `digitalRead(2)` returns `LOW` (or `0`), **not** `HIGH`!

3. Servo PWM Framing

- **Exam Question**: A micro-servo is commanded via `my_servo.write(0)`. What is the pulse width generated by the microcontroller?
- **Solution**: A 0° angle corresponds to a 1.0 ms **pulse** repeated every 20 ms (50 Hz). Neutral (90°) is 1.5 ms; full span (180°) is 2.0 ms.

4. 16-Bit Integer Overflow in Sensor Accumulators

- **Exam Question**: An engineer sums 1,000 analog readings using `int sum = 0;`. Why will this fail on an Arduino Uno?
- **Solution**: In AVR GCC on the ATmega328P, an `int` is 16-bit signed, with a maximum limit of +32,767. If each reading is ≈ 500 , the sum reaches 500,000, overflowing and wrapping into negative numbers! The accumulator must be declared as `long int` or `float`.